| | |
|---|---|
| **Started on** | Monday, 23 September 2024, 2:33 PM |
| **State** | Finished |
| **Completed on** | Monday, 23 September 2024, 4:08 PM |
| **Time taken** | 1 hour 34 mins |
| **Grade** | **41.50** out of 51.00 (**81**%) |

Question **1**

Partially correct

Mark 3.00 out of 5.00

The statements below relate to computer theory.

Indicate which of the statements are True and which are False.

| True | False | | | |
|---|---|---|---|---|
| ◉ ☑ | ○ ✖ | A Turing Machine can accept any language that could be accepted by a Finite Automaton. | ✔ | |
| ○ ☑ | ○ ✖ | The incompleteness theorem showed that there are problems for which no computations exist. | ✖ | |
| ○ ✖ | ◉ ☑ | NP-Complete problems can be solved in polynomial time. | ✔ | |
| ◉ ☑ | ○ ✖ | The execution of a set of instructions by an agent is called a computation. | ✔ | out of notes |
| ◉ ✖ | ○ ☑ | A tractable problem is one that cannot be solved in "reasonable" time. | ✖ | This would be an intractable problem. |

A Turing Machine can accept any language that could be accepted by a Finite Automaton.: True

The incompleteness theorem showed that there are problems for which no computations exist.: True

NP-Complete problems can be solved in polynomial time.: False

The execution of a set of instructions by an agent is called a computation.: True

A tractable problem is one that cannot be solved in "reasonable" time.: False

Question **2**

Correct

Mark 4.00 out of 4.00

Consider the statements below and say which of them are true or false.

| True | False | | | |
|---|---|---|---|---|
| ○ ✖ | ◉ ☑ | If $f(n) = 23n^2 + 7n + 12$ then $f \in O(n)$ | ✔ | |
| ◉ ☑ | ○ ✖ | If $f(n) = 4n^2 + 14n + 3$ then $f \in \Theta(n^2)$ | ✔ | |
| ◉ ☑ | ○ ✖ | If $f(n) = 16n^2 + 2n + 11$ then $f \in o(n^3)$ | ✔ | out of new complexity notes |
| ◉ ☑ | ○ ✖ | If $f(n) = n^2 + 27$ then $f \in \omega(n)$ | ✔ | |

If $f(n)=23n^2+7n+12$ then $f \in O(n)$

: False

If $f(n) = 4n^2 + 14n + 3$ then $f \in \Theta(n^2)$

: True

If $f(n) = 16n^2 + 2n + 11$ then $f \in o(n^3)$

: True

If $f(n) = n^2 + 27$ then $f \in \omega(n)$

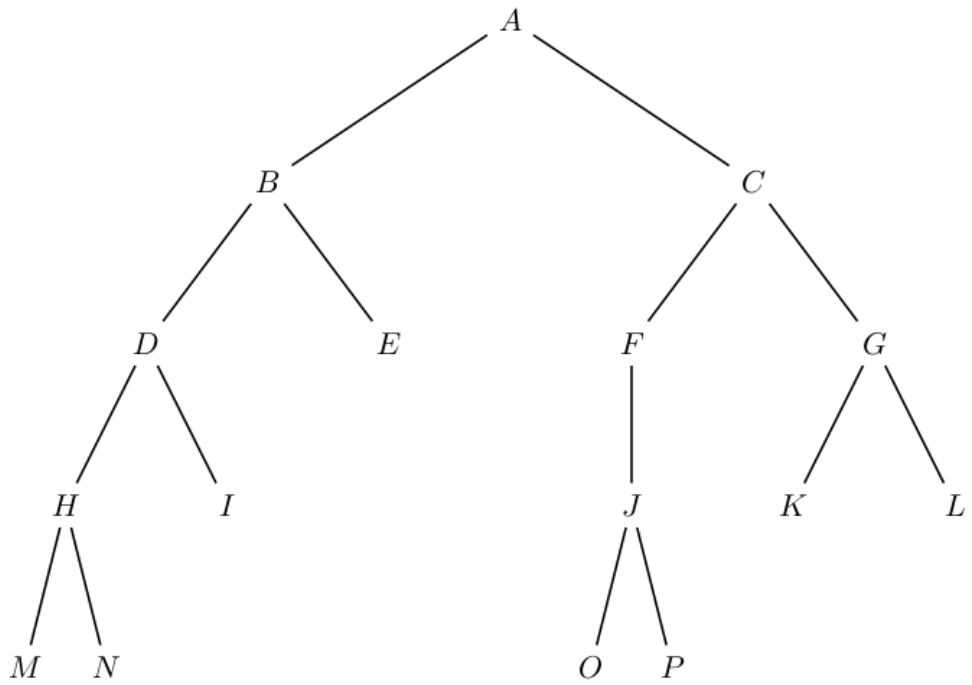
: True

Question **3**

Correct

Mark 1.00 out of 1.00

Consider the search tree shown below.




Suppose the tree is searched using the Breadth First Search tree search algorithm.

Note that expansion is done in a leftmost fashion. The child node shown to the left in the diagram will be the first one searched.

If the start node is A and the goal node is G, what will the frontier contain when the goal is found?

(Note: Only answers in the form X, Y, Z or X,Y,Z will be accepted. Case does not matter X = x.)

Answer: D,E,F ✔

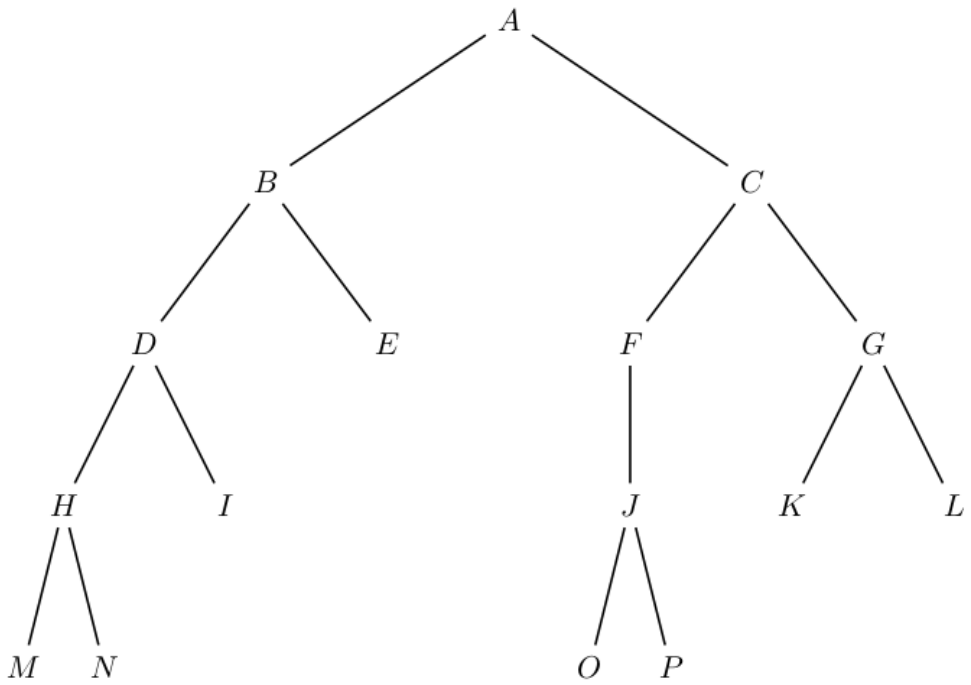
The correct answer is: D, E, F

Question **4**

Correct

Mark 1.00 out of 1.00

Consider the search tree shown below.




Suppose the tree is searched using the Depth First Search tree search algorithm.

Note that expansion is done in a leftmost fashion. The child node shown to the left in the diagram will be the first one searched.

If the start node is A and the goal node is M, what will the frontier contain when the goal is found?

(Note: Only answers in the form X, Y, Z or X,Y,Z will be accepted. Case does not matter X = x.)

Answer: N,I,E,C

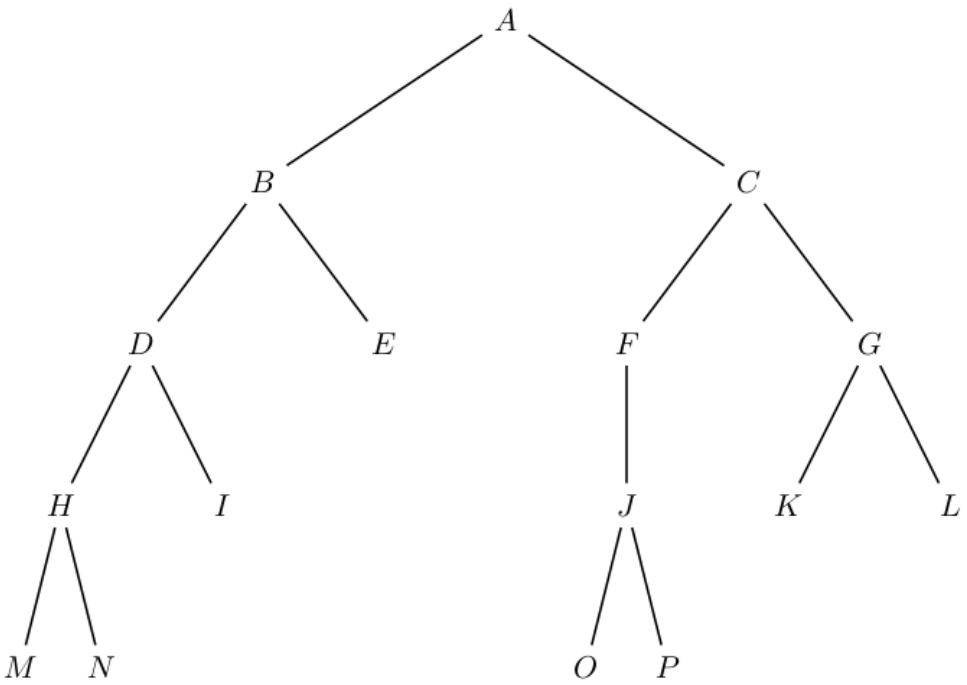
The correct answer is: N, I, E, C

Question **5**

Correct

Mark 3.00 out of 3.00

Consider the search tree shown below.




Suppose the tree is searched using the Iterative Deepening Search algorithm. Note that expansion is done in a leftmost fashion. The child node shown to the left in the diagram will be the first one searched.

Suppose that the start node is A and the goal node is O.

There will be 5 (0 to 4) ✔ phases of deepening.

When the goal is found the frontier will be P, G ✔ .

IDS will expand more ✔ nodes in total than a straightforward breadth first search would expand to find O.

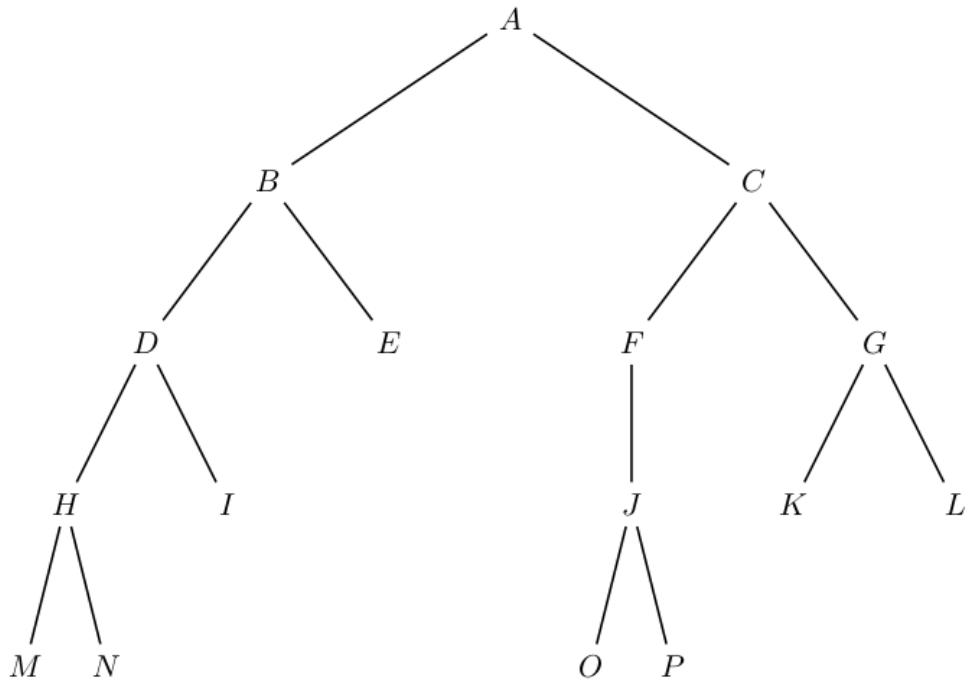
4 (0 to 3) | P, G, K, L | P, G, K

6 (0 to 5) | as many | fewer | 3 (0 to 2)

Your answer is correct.

The correct answer is:

Consider the search tree shown below.




Suppose the tree is searched using the Iterative Deepening Search algorithm. Note that expansion is done in a leftmost fashion. The child node shown to the left in the diagram will be the first one searched.

Suppose that the start node is A and the goal node is O.

There will be [5 (0 to 4)] phases of deepening.

When the goal is found the frontier will be [P, G].

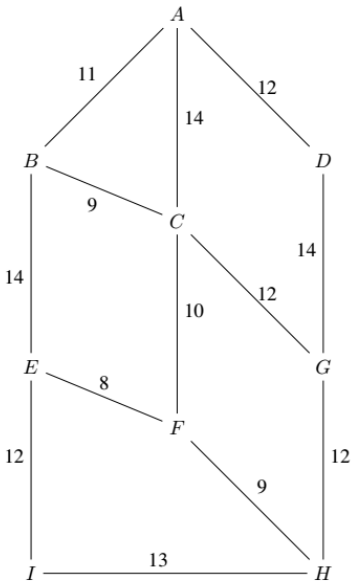
IDS will expand [more] nodes in total than a straightforward breadth first search would expand to find O.

Question **6**

Partially correct

Mark 2.00 out of 3.00

Consider the graph shown below.




Suppose uniform cost search is used to find the best path from A to H.

The frontier after A is expanded will be ['B', 'D', 'C'] ✔ .

The shortest path after expanding five nodes will be ACF cost 24 ✖ .

[redacted] ✔ nodes will be expanded before the search terminates and returns the shortest path.

| | | | | | |
|---|---|---|---|---|---|
| ['B', 'C', 'D'] | 4 | 9 | 6 | ABE cost 25 | 5 |

Your answer is partially correct.

You have correctly selected 2.
Enter the name of the file containing the inputucsct.txt
The graph is [['A', 'B', 11, 'C', 14, 'D', 12], ['B', 'A', 11, 'C', 9, 'E', 14], ['C', 'A', 14, 'B', 9, 'F', 10, 'G', 12], ['D', 'A', 12, 'G', 14], ['E', 'B', 14, 'F', 8, 'I', 12], ['F', 'C', 10, 'E', 8, 'H', 9], ['G', 'C', 12, 'D', 14, 'H', 12], ['H', 'F', 9, 'G', 12, 'I', 13], ['I', 'E', 12, 'H', 13]]
What is the goal nodeH
start node A
initial frontier ['A']
Looping -- step 1
Node to be expanded A
Current frontier []
Current possible paths []
Current expanded set ['A']
Children of node being expanded ['B', 11, 'C', 14, 'D', 12]
Child being considered B 11
Current frontier ['B']
possible paths [[11, 'B', 'AB']]
Child being considered C 14
Current frontier ['B', 'C']
possible paths [[11, 'B', 'AB'], [14, 'C', 'AC']]
Child being considered D 12

Current frontier ['B', 'D', 'C']
possible paths [[11, 'B', 'AB'], [12, 'D', 'AD'], [14, 'C', 'AC']]

Looping -- step 2
Node to be expanded B
Current frontier ['D', 'C']
Current possible paths [[12, 'D', 'AD'], [14, 'C', 'AC']]
Current expanded set ['A', 'B']
Children of node being expanded ['A', 11, 'C', 9, 'E', 14]
Child being considered A 11
have already seen the node
Current frontier ['D', 'C']
possible paths [[12, 'D', 'AD'], [14, 'C', 'AC']]
Child being considered C 9
have already seen the node
this node is already in the frontier
20
14
New route is longer - not added
Current frontier ['D', 'C']
possible paths [[12, 'D', 'AD'], [14, 'C', 'AC']]
Child being considered E 14
Current frontier ['D', 'C', 'E']
possible paths [[12, 'D', 'AD'], [14, 'C', 'AC'], [25, 'E', 'ABE']]

Looping -- step 3
Node to be expanded D
Current frontier ['C', 'E']
Current possible paths [[14, 'C', 'AC'], [25, 'E', 'ABE']]
Current expanded set ['A', 'B', 'D']
Children of node being expanded ['A', 12, 'G', 14]
Child being considered A 12
have already seen the node
Current frontier ['C', 'E']
possible paths [[14, 'C', 'AC'], [25, 'E', 'ABE']]
Child being considered G 14
Current frontier ['C', 'E', 'G']
possible paths [[14, 'C', 'AC'], [25, 'E', 'ABE'], [26, 'G', 'ADG']]

Looping -- step 4
Node to be expanded C
Current frontier ['E', 'G']
Current possible paths [[25, 'E', 'ABE'], [26, 'G', 'ADG']]
Current expanded set ['A', 'B', 'D', 'C']
Children of node being expanded ['A', 14, 'B', 9, 'F', 10, 'G', 12]
Child being considered A 14
have already seen the node
Current frontier ['E', 'G']
possible paths [[25, 'E', 'ABE'], [26, 'G', 'ADG']]
Child being considered B 9
have already seen the node
Current frontier ['E', 'G']
possible paths [[25, 'E', 'ABE'], [26, 'G', 'ADG']]
Child being considered F 10
Current frontier ['F', 'E', 'G']
possible paths [[24, 'F', 'ACF'], [25, 'E', 'ABE'], [26, 'G', 'ADG']]
Child being considered G 12
have already seen the node
this node is already in the frontier
26

26
new path through this node is shorter
Old path removed
Current frontier ['F', 'E', 'G']
possible paths [[24, 'F', 'ACF'], [25, 'E', 'ABE'], [26, 'G', 'ACG']]

Looping -- step 5
Node to be expanded F
Current frontier ['E', 'G']
Current possible paths [[25, 'E', 'ABE'], [26, 'G', 'ACG']]
Current expanded set ['A', 'B', 'D', 'C', 'F']
Children of node being expanded ['C', 10, 'E', 8, 'H', 9]
Child being considered C 10
have already seen the node
Current frontier ['E', 'G']
possible paths [[25, 'E', 'ABE'], [26, 'G', 'ACG']]
Child being considered E 8
have already seen the node
this node is already in the frontier
32
25
New route is longer - not added
Current frontier ['E', 'G']
possible paths [[25, 'E', 'ABE'], [26, 'G', 'ACG']]
Child being considered H 9
Current frontier ['E', 'G', 'H']
possible paths [[25, 'E', 'ABE'], [26, 'G', 'ACG'], [33, 'H', 'ACFH']]

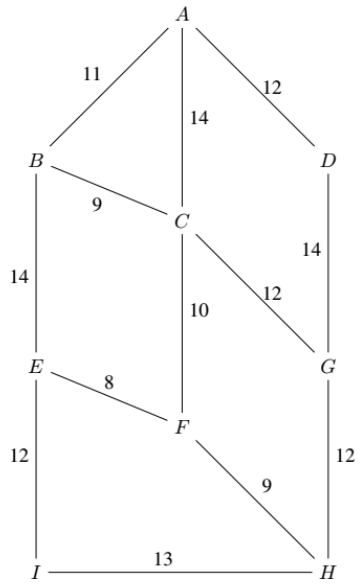
Looping -- step 6
Node to be expanded E
Current frontier ['G', 'H']
Current possible paths [[26, 'G', 'ACG'], [33, 'H', 'ACFH']]
Current expanded set ['A', 'B', 'D', 'C', 'F', 'E']
Children of node being expanded ['B', 14, 'F', 8, 'I', 12]
Child being considered B 14
have already seen the node
Current frontier ['G', 'H']
possible paths [[26, 'G', 'ACG'], [33, 'H', 'ACFH']]
Child being considered F 8
have already seen the node
Current frontier ['G', 'H']
possible paths [[26, 'G', 'ACG'], [33, 'H', 'ACFH']]
Child being considered I 12
Current frontier ['G', 'H', 'I']
possible paths [[26, 'G', 'ACG'], [33, 'H', 'ACFH'], [37, 'I', 'ABEI']]

Looping -- step 7
Node to be expanded G
Current frontier ['H', 'I']
Current possible paths [[33, 'H', 'ACFH'], [37, 'I', 'ABEI']]
Current expanded set ['A', 'B', 'D', 'C', 'F', 'E', 'G']
Children of node being expanded ['C', 12, 'D', 14, 'H', 12]
Child being considered C 12
have already seen the node
Current frontier ['H', 'I']
possible paths [[33, 'H', 'ACFH'], [37, 'I', 'ABEI']]
Child being considered D 14
have already seen the node
Current frontier ['H', 'I']
possible paths [[33, 'H', 'ACFH'], [37, 'I', 'ABEI']]

Child being considered H 12
have already seen the node
this node is already in the frontier
38
33
New route is longer - not added
Current frontier ['H', 'I']
possible paths [[33, 'H', 'ACFH'], [37, 'I', 'ABEI']]

Looping -- step 8
Node to be expanded H
Current frontier ['I']
Current possible paths [[37, 'I', 'ABEI']]
Current expanded set ['A', 'B', 'D', 'C', 'F', 'E', 'G', 'H']
goal found
H
path to goal is ACFH
Solution length is 33

The correct answer is:
Consider the graph shown below.




Suppose uniform cost search is used to find the best path from A to H.

The frontier after A is expanded will be [['B', 'D', 'C']].

The shortest path after expanding five nodes will be [ABE cost 25].

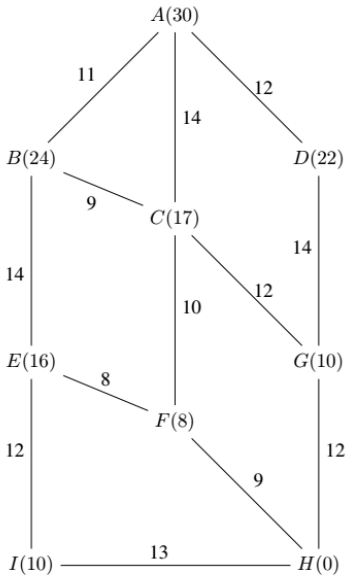
[8] nodes will be expanded before the search terminates and returns the shortest path.

Question **7**

Partially correct

Mark 2.00 out of 3.00

Consider the search graph shown below.




Here the numbers on the edges are g(n) and the numbers in brackets next to the node names are h(n).

Suppose A* search is used starting at node A and the goal is node H.

After node A has been expanded the node at the front of the priority queue is C ✔ .

After the third node has been expanded in the search there are 5 ✔ nodes in the frontier.

After the goal has been found the node that appears at the front of the priority queue (the frontier) is B ✖ .

G 4 D E 6 3

Your answer is partially correct.

You have correctly selected 2.
The graph is [['A', 30, 'B', 11, 'C', 14, 'D', 12], ['B', 24, 'A', 11, 'C', 9, 'E', 14], ['C', 17, 'A', 14, 'B', 9, 'F', 10, 'G', 12], ['D', 22, 'A', 12, 'G', 14], ['E', 16, 'B', 14, 'F', 8, 'I', 12], ['F', 8, 'C', 10, 'E', 8, 'H', 9], ['G', 10, 'C', 12, 'D', 14, 'H', 12], ['H', 0, 'F', 9, 'G', 12, 'I', 13], ['I', 10, 'E', 12, 'H', 13]]
What is the goal nodeH
start node and its h A 30
initial frontier ['A']
initial possible path [[30, 0, 30, 'A', 'A']]
Looping -- step 1
Node to be expanded A
Current frontier []
Current possible paths []
Current expanded set ['A']
Children of node being expanded [30, 'B', 11, 'C', 14, 'D', 12]
1 7
Child being considered B 11
The heuristic h(n) for this child node 24
Current frontier ['B']
possible paths [[35, 11, 24, 'B', 'AB']]
3 7

Child being considered C 14
The heuristic h(n) for this child node 17
Current frontier ['C', 'B']
possible paths [[31, 14, 17, 'C', 'AC'], [35, 11, 24, 'B', 'AB']]
5 7
Child being considered D 12
The heuristic h(n) for this child node 22
Current frontier ['C', 'D', 'B']
possible paths [[31, 14, 17, 'C', 'AC'], [34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB']]

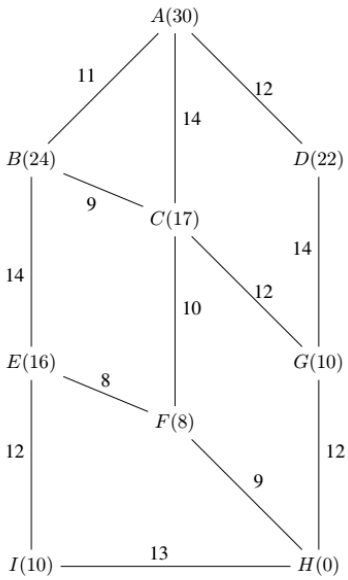
Looping -- step 2
Node to be expanded C
Current frontier ['D', 'B']
Current possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB']]
Current expanded set ['A', 'C']
Children of node being expanded [17, 'A', 14, 'B', 9, 'F', 10, 'G', 12]
1 9
Child being considered A 14
have already seen the node
Current frontier ['D', 'B']
possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB']]
3 9
Child being considered B 9
have already seen the node
this node is already in the frontier
40
35
New route is longer - not added
Current frontier ['D', 'B']
possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB']]
5 9
Child being considered F 10
The heuristic h(n) for this child node 8
Current frontier ['F', 'D', 'B']
possible paths [[32, 24, 8, 'F', 'ACF'], [34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB']]
7 9
Child being considered G 12
The heuristic h(n) for this child node 10
Current frontier ['F', 'D', 'B', 'G']
possible paths [[32, 24, 8, 'F', 'ACF'], [34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG']]

Looping -- step 3
Node to be expanded F
Current frontier ['D', 'B', 'G']
Current possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG']]
Current expanded set ['A', 'C', 'F']
Children of node being expanded [8, 'C', 10, 'E', 8, 'H', 9]
1 7
Child being considered C 10
have already seen the node
Current frontier ['D', 'B', 'G']
possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG']]
3 7
Child being considered E 8
The heuristic h(n) for this child node 16
Current frontier ['D', 'B', 'G', 'E']
possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG'], [48, 32, 16, 'E', 'ACFE']]
5 7
Child being considered H 9
The heuristic h(n) for this child node 0

Current frontier ['H', 'D', 'B', 'G', 'E']
possible paths [[33, 33, 0, 'H', 'ACFH'], [34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG'], [48, 32, 16, 'E', 'ACFE']]

Looping -- step 4
Node to be expanded H
Current frontier ['D', 'B', 'G', 'E']
Current possible paths [[34, 12, 22, 'D', 'AD'], [35, 11, 24, 'B', 'AB'], [36, 26, 10, 'G', 'ACG'], [48, 32, 16, 'E', 'ACFE']]
Current expanded set ['A', 'C', 'F', 'H']
goal found
H
path to goal is ACFH
Solution f is 33

The correct answer is:
Consider the search graph shown below.




Here the numbers on the edges are g(n) and the numbers in brackets next to the node names are h(n).

Suppose A* search is used starting at node A and the goal is node H.

After node A has been expanded the node at the front of the priority queue is [C].

After the third node has been expanded in the search there are [5] nodes in the frontier.

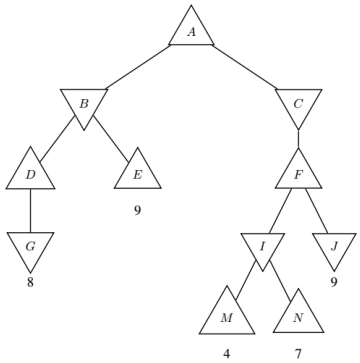
After the goal has been found the node that appears at the front of the priority queue (the frontier) is [D].

Question **8**

Correct

Mark 1.00 out of 1.00

Consider the game tree shown below.




If the minimax algorithm is applied to the tree then the utility values of B, D, A, C, F (in that order) would be

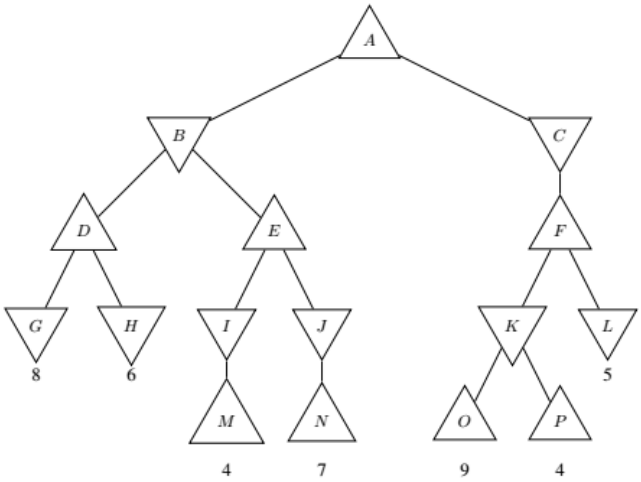
Answer: 8,8,9,9,9

The correct answer is: 8, 8, 9, 9, 9

Question **9**

Correct

Mark 1.00 out of 1.00




Consider the game tree shown below.

Here upward pointing triangles are max nodes and downward point triangles are min nodes.

If alpha beta pruning is applied to this tree then:

- [x] a. No pruning is done ✔
- [ ] b. Nodes P and L are pruned.
- [ ] c. Node J is pruned
- [ ] d. Node P is pruned
- [ ] e. Node L is pruned

Your answer is correct.

The correct answer is:
No pruning is done

Question **10**

Correct

Mark 1.00 out of 1.00

Select the statements regarding alpha-beta pruning below that are true

- [ ] a. Alpha-beta pruning may not return the same value as minimax.
- [x] b. The running time of alpha-beta pruning (as measured in seconds) is generally less than that of minimax for a given depth ✔
- [ ] c. Alpha-beta pruning will prune the same number of subtrees, regardless of the order in which successor states are expanded

Your answer is correct.

The correct answer is:

The running time of alpha-beta pruning (as measured in seconds) is generally less than that of minimax for a given depth

Question **11**

Correct

Mark 1.00 out of 1.00

When we talk about the "basic operation" in algorithm analysis which of the following do we mean?

- ○ a. Any operation on an array element
- ○ b. The operations of addition, subtraction, multiplication and division
- ◉ c. The operation[s] that describes the essential task that any algorithm needs to do to solve the given problem. ✔
- ○ d. The operation[s] that an algorithm does to solve the problem
- ○ e. Operations for opening, writing to and closing files.

Your answer is correct.

The correct answer is:
The operation[s] that describes the essential task that any algorithm needs to do to solve the given problem.

Question **12**

Complete

Mark 6.00 out of 6.00

Design an algorithm that will determine whether the first number in a list of numbers occurs again in the list.

Analyse your algorithm.

Consider the best and worst cases (if applicable).

```
checkForReoccuring(list,n)
Input:list,n where list is the list of numbers with indexes 0...(n-1)
Output:true if first element in list occurs again in the list, else false
value = list[0]
currentIndex = 1
while (currentIndex>n):
   if (list[currentIndex]==value):
      return true
   currentIndex = currentIndex+1
return true
```

Best case:
If list[1] is equal to list[0], the same value is at index 0 and 1, then the algorithm has to do 1 comparison before returning true. Thus it is O(1)

Worst case:
If the list has distinct values, then no other element in list will equal list[0]. Thus the while loop will run n-1 times[-1 since we start currentIndex at 1], before exiting the while loop and returning false. Hence the time complexity of the worst case is O(n)

Comment:

Question **13**

Complete

Mark 10.00 out of 10.00

Consider the algorithm given below.

```
00  Algorithm funnySearch(myList, n)
    Input:  myList where myList is a sorted array
       with n entries (indexed 0...n-1).
    key is the number being searched for and assumed to be in the list.
    Output:  The position of the number in the list.
01      q2 ← ⌊n/2⌋
02      q1 ← ⌊n/4⌋
03      q3 ← q1 × 3
04      If key < myList[q2]
05
06          If key < myList[q1]
07
08              If
09
10          Else
11              l = q1
12              r = q2
13      Else
14          If key > myList[q3]
15          Then
16              l = q3 + 1
17              r = n - 1
18          Else
19              l = q2 + 1
20              r = q3
21  For j from l to r
22      If myList[j] = key
23      Then
24          Return j
```

Explain how the algorithm solves the problem that it was designed to solve. (3 marks)

Argue that the algorithm will always find the key (provided the initial assumption is true). (3 marks)

Analyse the algorithm in terms of its **average case running time**. (4 Marks)

1)Given we have a sorted list. i.e all the numbers are sorted. This algorithm finds the key, which is assumed to be in the list, by check which quarter of data the key is found in. q1 represents the first 25% of the list, q2 the next 25%, and q3 the next 25%
Thus we have this:
q1:0->25% of the list
q2:25%->50% of the list
q3:50%->75% of the list

the first if statement basically finds between which 2 quartiles of the list the key is in.
We set the start of the search to be the start of the lower quartile + 1[l variables], and set the end of the search to be the start of the next quartile[r variable].

We then execute a for loop from l to r, that will check the numbers in this range and compare it to our key. And since the list is sorted, we know that the key is in this range. Thus we have found the position of the key in the list and can return the index.

2) Assume that the algorithm works for lists of length k

Let us show that it works for lists of length k+1.

Assume the list of length k+1 is sorted.
When we do the same algorithm, it will result in quartiles being created. These quartiles are equivalent/ if not the same as a list with k elements

FLOOR((k+1)/4) = FLOOR((k)/4

Thus the problem is still solved a list with k elements, thus the algorithm works for lists with length k, thus it works for lists with length k+1 and is correct.

3)Assume the key has an equal chance to be any number in the list, thus 1/n
Assume that the list is sorted

After we execute the first if statement, we effectively reduce our search space to be between two quartiles, thus n/4 of the data

We thus need to consider the for loop from l to r

Since each element has an equal chance of being the key, and we do 1 comparison each time we consider an element,we effectively have the following sum:

sum from l to r: 1/n * (1+2+3+4+...+n)=
NB: we do 1 comparison if my myList[l] is the key, 2 comparisons if myList[l+1] is the key...

Thus we have 1/n * (n(n-1))/2
Which equals (n-1)/2
Which means an average case of O(n)

Comment:

Question **14**

Correct

Mark 1.00 out of 1.00

Which of the statements below **best** explains why linear search (where the key must be in the list) is considered **optimal**.

- a. Linear search works by comparing the key to the numbers in the list
- b. The work done in the algorithm's worst case matches the work required to solve the problem. ✔
- c. It is the best algorithm we have found.
- d. No other algorithm is faster than linear search.

Your answer is correct.

The correct answer is:
The work done in the algorithm's worst case matches the work required to solve the problem.

Question **15**

Correct

Mark 1.00 out of 1.00

Suppose that the bisection search algorithm (where the key must be in the list) is run on the sorted list given below.

[12, 13, 23, 34, 45, 56, 76, 87, 93, 111, 123, 134, 157, 234, 342, 347, 369]

How many comparisons between key and an element in the list would be required if key = 12?

**Algorithm 3** bisectionSearch(myList,n,key)

*Input:* $myList, n, key$ where $myList$ is an array with $n$ entries (indexed $0 \ldots (n-1)$), and $key$ is the item sought.
The values stored in $myList$ are such that
$myList[0] \leq myList[1] \leq \ldots \leq myList[n-2] \leq myList[n-1]$
*Output:* $mid$ the location of $key$ in $myList$.

```
01 low ← 0
02 high ← n − 1
03 mid ← ⌊(low + high)/2⌋
04 While key ≠ myList[mid]
05    If key < myList[mid]
06       Then high ← mid − 1
07       Else low ← mid + 1
08    mid ← ⌊(low + high)/2⌋
09 return mid
```

Answer: 7 ✔

in while comparing in position 8 , then inside loop checking to see which sublist to consider, 2 comparisons

in while comparing in position 3 , then inside loop checking to see which sublist to consider, 2 comparisons
in while comparing in position 1 , then inside loop checking to see which sublist to consider, 2 comparisons
in while comparing in position 0, 1 comparison

The correct answer is: 7

Question **16**

Correct

Mark 1.00 out of 1.00

Suppose that the bisection search where the key may not be in the list is run on the list below.

[12, 13, 23, 34, 45, 56, 76, 87, 93, 111, 123, 134, 157, 234, 342, 347, 369]

How many comparisons between the key and list elements would be required to find that key = 88 is not in the list?

**Algorithm 4** bisectionSearch(myList,n, key)

*Input:* $myList, n, key$ where $myList$ is an array with $n$ entries (indexed $0 \ldots (n-1)$), and $key$ is the item sought.
The values stored in $myList$ are such that
$myList[0] \leq myList[1] \leq \ldots \leq myList[n-2] \leq myList[n-1]$
*Output:* $mid$ the location of $key$ in $myList$ ($-1$ if $key$ is not found).

```
01 low ← 0
02 high ← n − 1
03 found ← False
04 While low ≤ high and found = False
05    mid ← ⌊(low + high)/2⌋
06    If key = myList[mid]
07      Then found ← True
08      Else
09         If key < myList[mid]
10           Then high ← mid − 1
11           Else low ← mid + 1
12 If found = False
13    Then mid ← −1
14 Return mid
```

Answer: 10 ✔

The correct answer is: 10

Question **17**

Correct

Mark 1.00 out of 1.00

Imagine we have an arbitrarily large list that is in sorted order.

Which of the sorting algorithms below has the best asymptotic performance for such input?

- a. Bubblesort
- b. Selection sort
- c. Insertion sort ✔
- d. Max sort
- e. Mergesort

Your answer is correct.

The correct answer is:
Insertion sort

Question **18**

Complete

Mark 0.50 out of 4.00

Derive a divide and conquer algorithm to return the smallest number in a list of numbers.

getSmallest(list,n,left,right)
*Input:list,n,minimum where list is the list of numbers with indexes 0...(n-1) and left is the left endpoint and right is the right endpoint*
*Output:smallest element*

if (right-left>1):

```
smallest(list,l,r)
if l = r return list[l]
else
mid = (l+r)/2
a = smallest(list, l, mid)
b = smallest(list,mid+1, r)
return minimum(a,b)
```

Comment:

Question **19**

Incorrect

Mark 0.00 out of 1.00

Suppose the list of numbers as shown below was sorted using Insertion Sort:

21, 23, 45, 13, 15, 19, 34, 32, 41

How many comparisons would be made to end up with the partially sorted list as below.

13, 15, 21, 23, 45, 19, 34, 32, 41

That is, the numbers up to the 45 are in sorted order.

**Algorithm 5** insertionSort($myList$, $n$)

The insertion sort algorithm
*Input:* $myList$, $n$ where $myList$ is an array with $n$ entries (indexed $0 \ldots n-1$)
*Output:* $myList$ where the values in $myList$ are such that
$myList[0] \leq myList[1] \leq \ldots \leq myList[n-2] \leq myList[n-1]$

```
01  For i from 1 to n − 1
02      x ← myList[i]
03      j ← i − 1
04      While (j >= 0) and myList[j] > x
05          myList[j + 1] ← myList[j]
06          j ← j − 1
07      myList[j + 1] ← x
08  Return myList
```

Answer: 15 ✖

The correct answer is: 9

Question **20**

Correct

Mark 1.00 out of 1.00

Suppose mergesort is used to sort the list of numbers show below.

21, 23, 45, 13, 15, 19, 34, 32, 41

The initial call to the mergesort will be

mergesort(0,8)

Is the statement below True or False:

**In this case, there will be more recursive calls to sort the left sublist of the original list than there will be recursive calls to sort the right sublist of the original list.**

**Algorithm 8** mergeSort($left$, $right$)

The mergesort algorithm
*Input:* $left$, $right$ where $left$ and $right$ are indices into an array $myList$ of $n$ entries (initially indexed $0 \ldots n-1$)
*Output:* a portion of the array $myList$ where the values in $myList$ are such that $myList[left] \leq myList[left+1] \leq \ldots \leq myList[right-1] \leq myList[right]$

```
01  IF right − left > 0
02     Then
03        mid ← ⌊(left + right)/2⌋
04        mergeSort(left,mid)
05        mergeSort(mid + 1,right)
06        For i from mid down to left
07           temp[i] ← myList[i]
08        For j from mid + 1 to right
09           temp[right + mid + 1 − j] ← myList[j]
10        i ← left
11        j ← right
12        For k from left to right
13           If temp[i] < temp[j]
14              Then
15                 myList[k] ← temp[i]
16                 i ← i + 1
17              Else
18                 myList[k] ← temp[j]
19                 j ← j − 1
```

Select one:

- True ✔
- False

The left sublist will be longer and so will require more calls.

The correct answer is 'True'.

Question **21**

Incorrect

Mark 0.00 out of 1.00

Suppose that quicksort is used to sort the list of numbers shown below.

21, 23, 45, 13, 15, 19, 34, 32, 41

The first call to the quicksort algorithm will be

quicksort(0,8)

Which number will be the fourth number to be moved into its correct position by a partition operation?

**Algorithm 10** quickSort($left$, $right$)

The quicksort algorithm
*Input:* $left$, $right$ where $left$ and $right$ are indices into an array of $n$ entries (initially indexed $0 \ldots n-1$)
*Output:* a portion of the array $myList$ where the values in $myList$ are such that $myList[left] \leq myList[left+1] \leq \ldots \leq myList[right-1] \leq myList[right]$

```
01  IF right > left
02   Then
03     v ← myList[right]
04     i ← left
05     j ← right
06     While i < j
07       While myList[i] < v
08         i ← i + 1
09       While j > i and myList[j] ≥ v
10         j ← j − 1
11       If j > i
12         Then
13           t ← myList[i]
14           myList[i] ← myList[j]
15           myList[j] ← t
16         ELSE t ← myList[i]
17           myList[i] ← myList[right]
18           myList[right] ← t
19     quickSort(left,i − 1)
20     quickSort(i + 1,right)
```

Answer: 15 ✖

The correct answer is: 13

◄ Quiz 5 Hashing and Red Black Trees

Jump to...

November Exam ►